

Fine-Grained Dynamic Update Controller Synthesis

ANONYMOUS AUTHOR(S)

Verified old and new controllers leave their connecting update unchecked. Components may change at different times, under requirements with different lifetimes. Fine-Grained Dynamic Update Controller Synthesis (FG-DUCS) expresses these choices as a contract between fixed controllers. A finite reachability game and lazy exploration return a policy, progress rank and handover targets, or a losing-region certificate. On 27 contracts from 9 DUCS-derived inputs, lazy exploration decides 26 within a 1,200 s tool-run limit and 64 GiB Java heap; full-graph exploration in the same implementation decides 14. Witnesses separate local from simultaneous transfer and lazy from full-graph exploration. Alternative priorities can be faster and one contract is unrealizable. All three on-the-fly variants decide only 20/48 Travel settings, versus 18 for full-graph exploration. Guarantees cover the supplied game; execution and requirement interpretation have separate assumptions.

CCS Concepts: • **Software and its engineering** → **Software evolution**.

Additional Key Words and Phrases: dynamic software updating, discrete-event controller synthesis, supervisory control, on-the-fly synthesis

ACM Reference Format:

Anonymous Author(s). 2027. Fine-Grained Dynamic Update Controller Synthesis. In *Proceedings of The ACM International Conference on the Foundations of Software Engineering (FSE 2027)*. ACM, New York, NY, USA, 20 pages.

1 Introduction

Updating software while it remains in service must preserve work already in progress [3, 6, 12, 27]. A production cell may need new station software while a workpiece is in the line; a workflow engine may need new requirements while cases are running. Stopping and emptying the system would interrupt that work. Verifying the old and new controllers separately leaves their connecting update unchecked: components change at different times, requirements begin or expire, and the update can be requested in any state reached during normal operation.

Kubernetes rollouts and MongoDB’s Community-to-Enterprise replica-set migration replace replicas in stages, waiting for readiness or rejoining [22, 28]. These practices motivate contracts that distinguish local replacement, possible outcomes and requirement lifetimes; Section 6.2 tests constructed abstractions of these mechanisms.

Dynamic Update Controller Synthesis (DUCS) synthesizes update and subsequent control [23, 24]. Its inputs relate old to new environment states and stop/start whole requirement sets; intermediate stages can be encoded in the new environment. FG-DUCS instead takes local component changes (transfers) and individual requirement lifetimes, and synthesizes their interleaving with normal operations.

Old and chosen new controllers remain fixed. The engineer supplies allowed transfers and outcomes, requirements before/during/after transfer, permitted boundary orders and handover targets. FG-DUCS returns a policy selecting operations from the current state, or evidence that no policy satisfies this contract. Every reachable old state and every possible outcome must finish safely.

Our contributions are this contract and finite reachability game; a terminating, sound and complete lazy procedure with success/failure evidence checkable relative to shared successor and end-point predicates; and checks and measurements addressing the obstacles below. On 27 contracts from 9 adapted inputs, lazy exploration decides 26 within budget, versus 14 for full construction;

failures and faster alternatives remain reported. Game guarantees concern active monitors and finite completion; requirement interpretation and execution need Section 5's additional conditions.

2 Motivating Example

Consider a production cell with two stations, *A* and *B*, passing one workpiece between them [18, 24]. Both stations receive new versions, with inspection added at *B*. The old requirement (rule) forbids inspection; the new quality requirement requires inspection before the workpiece leaves *B*. The upgrade must continue the work in progress, from either station holding the workpiece.

Each station is replaceable only while empty. Both are never empty together, so replacing everything at once has no safe moment. The policy in Figure 1 replaces the empty station first and uses ordinary movement to free the other. It shows one way to finish from either starting situation.

The contract requires the new requirement to start before the old one stops, leaving no gap. On the upper path, *B* holds the workpiece when the new requirement starts: inspection becomes pending. Moving the workpiece out would violate the new requirement; inspecting it would violate the old requirement. Replacing the empty *A* remains possible, but cannot clear this inspection obligation. The shown policy stops the old requirement, inspects, then finishes replacement. On the lower path, the workpiece leaves *B* before the new requirement starts, so the policy finishes without inspection. Handover—passing control to the new controller—requires no inspection to be pending.

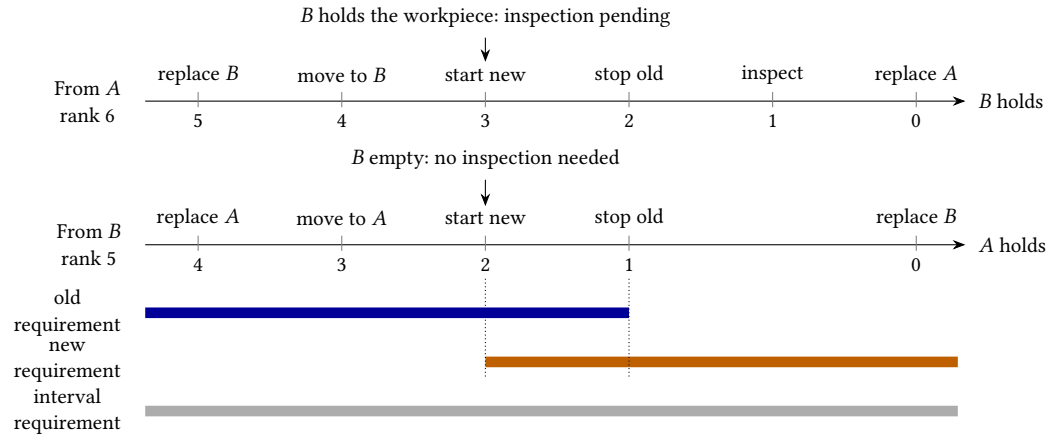


Fig. 1. A completing policy from both starting situations. Start/stop concern the new/old requirement. Annotations describe requirement activation; numbers below events give the remaining-step ranks after those events. Bars show requirement lifetimes, not elapsed time. Both paths finish with no inspection pending.

Inspection leaves the workpiece at *B* and remains enabled after it has been inspected. Repeating it forever would obey both currently active requirements but never complete replacement. A correct policy must choose a finite route to handover, not just avoid requirement violations. Other finite routes are possible: ordinary movements permit either fixed replacement order.

The engineer provides the verified controllers, both station versions and the conditions below; FG-DUCS returns the missing policy or evidence that none exists. Section 3.3 checks the policy; S1 gives the finite LTSs.

Replacement	Replace a station only when empty.
Old requirement	Forbid inspection until this requirement stops.
New requirement	Inspect before leaving B ; start this requirement before stopping the old one.
Interval requirement	Keep exactly one workpiece throughout the update.
Handover	Finish all changes with no inspection pending and a matching new-controller state.

Three obstacles organize the rest of the paper. **L1: local changes may succeed when simultaneous replacement cannot**; Section 3.3 proves this existence result and RQ2 checks granularity witnesses. **L2: every starting state and outcome must finish safely**; Theorem 4.3 proves the policy guarantee and RQ1 checks its implementation. **L3: additional components can multiply the intermediate version combinations**; Section 4.3 proves an exploration gap on a constructed family, and RQ3 measures savings and limits on the adapted models and scaling families.

The cell's events are all controllable. Adversarial outcomes in L2 appear in Railcab's transfers (Section 6.2); the scaling cost in L3 appears in Section 4.3's component family, not in this cell.

3 Defining an Update Contract

An update contract fixes the component transfers, active requirements and handover conditions. The game makes every permitted event and its outcomes explicit.

Notation	Meaning
g_i, ρ_i	Local transfer relation and its one-shot transfer event
$R^{\text{old}}, R^{\text{new}}, R^{\text{upd}}$	Old, new and interval requirements
t_r, Z_{load}	Monitor initializer and supplied handover targets
$Q_0, \text{Post}, Q_{\text{goal}}$	All entry states, complete successors and goals
$\sigma, \text{rk}, \kappa$	Policy, progress rank and handover map

3.1 Supplied Choices

Finite LTSs and safety monitors. An LTS $\mathcal{L} = (S, \Sigma_{\mathcal{L}}, \delta, s^0)$ has nondeterministic transitions $\delta : S \times \Sigma_{\mathcal{L}} \rightarrow 2^S$. Partition events as $\Sigma = \Sigma_{\text{uc}} \uplus \Sigma_{\text{c}}$: controllers may disable controllables but cannot select outcomes or suppress uncontrollable (UC) events. A synchronous-product event requires a transition in every component whose alphabet contains it; other components retain their state [19]. A monitor is an automaton that checks a requirement as events occur. Requirement r has a total deterministic monitor $t_r = (M_r, \Sigma, \eta_r, m_r^0, F_r)$ with absorbing errors for violating prefixes [1, 17]. The empty monitor product has one non-error state. Monitors observe labels; policies also observe successor component states.

Components and transfers. For each $i \in I$, let $e_i^{\text{old}}, e_i^{\text{new}}$ be the pre-update and post-update components and let $g_i \subseteq S_{e_i^{\text{old}}} \times S_{e_i^{\text{new}}}$ be their state-transfer relation. Define $g_i(s) = \{s' \mid (s, s') \in g_i\}$. The supplied g_i includes every modeled transformation outcome. Let $E^v = \parallel_{i \in I} e_i^v$.

Requirements and activation. Let pairwise-disjoint $R^{\text{old}}, R^{\text{new}}, R^{\text{upd}}$ contain the pre-update, post-update, and update-interval safety requirements, respectively. For each $r \in R^{\text{new}} \cup R^{\text{upd}}$, the input supplies a partial *monitor-initialization map* t_r from version-tagged component tuples to $M_r \setminus F_r$. Its domain specifies where activation is allowed; in the cell, initialization records whether inspection is pending. Residual soundness (RS, Section 5.1) relates the installed state's continuations to the declared activation histories. For $r \in R^{\text{upd}}$, $\text{dom } t_r$ includes every reachable old component state.

Endpoints and handover targets. For $v \in \{\text{old}, \text{new}\}$, the supplied finite controller C^v preserves uncontrollables and makes $E^v \parallel C^v$ deadlock-free and satisfy R^v . Hiding controller auxiliary events

leaves observation alphabet $\bigcup_i \Sigma_{e_i}^v$; the union over both versions is the ordinary alphabet Σ_N . Write Reach for initial-state reachability and

$$\text{CL}^v = E^v \parallel C^v \parallel \parallel_{r \in R^v} t_r.$$

Monitors are passive: every endpoint transition originates in $E^v \parallel C^v$; a monitor only updates on that label and adds no enabled event. The input supplies nonempty, checked targets $Z_{\text{load}} \subseteq \text{Reach}(\text{CL}^{\text{new}})$ with component, controller and new-monitor states. FG-DUCS keeps the supplied new controller fixed; the evaluation admits as targets all of $\text{Reach}(\text{CL}^{\text{new}})$. For a supplied maximally permissive controller this is its reachable winning region, equal to the full region only if all winning states are represented and reachable.

Update commands and order. The set of update events, each executed exactly once, is

$$\Sigma_{\text{upd}} = \{\rho_i \mid i \in I\} \cup \{\text{stop}(r) \mid r \in R^{\text{old}}\} \cup \{\text{start}(r) \mid r \in R^{\text{new}}\}. \quad (1)$$

These distinct controllable events transfer a component, stop an old monitor or start a new one, respectively. They are disjoint from ordinary events Σ_N : $\Sigma = \Sigma_N \uplus \Sigma_{\text{upd}}$ and $\Sigma_{\text{upd}} \subseteq \Sigma_c$. Each transfer event ρ_i is one atomic model transition: the controller chooses the update, but cannot choose its g_i outcome. They require uncontrollable quiescence (Section 3.2).

An optional strict partial order $< \subseteq \Sigma_{\text{upd}} \times \Sigma_{\text{upd}}$ expresses precedence among update operations. For a pending-event set P , event a is order-enabled exactly when $a \in P$ and every $a' < a$ has left P . Monitors self-loop on update events that they do not specify.

Eligible contracts. Eligibility requires finite, consistently typed states/events and consistent controllability; total deterministic monitors with absorbing errors; well-typed transfers and non-error initializers covering every interval entry; and strict partial-order dependencies. Both endpoints must be safe, deadlock-free and uncontrollable-preserving, with controller alphabets containing every environment label; handover targets must be nonempty and reachable. Separately, the modeler declares admissible activation histories $H_r(\xi)$ and justifies RS (Section 5.1). Histories parameterize interpretation, not the game solver; eligibility does not check their completeness. Interval-wide obligations belong to R^{upd} and post-handover obligations to R^{new} ; $\text{start}(r^{\text{new}}) < \text{stop}(r^{\text{old}})$ enforces overlap.

3.2 States, Transitions, and Completion

The induced game. States and all entry states. An update state $q = (\xi, \mu, P)$ records each component's version/state $\xi(i) = (v, s)$ with $s \in S_{e_i}^v$, each monitor's state $\mu(t_r) \in M_r \cup \{\perp\}$ ($\perp$ means inactive), and pending commands $P \subseteq \Sigma_{\text{upd}}$. Here $\rho_i \in P$ exactly when component i is old. Old monitors remain active until their stop leaves P ; new monitors become active when their start leaves P ; update monitors are always active.

Admission $\text{Init}(z)$ inherits z 's component and old-monitor states, initializes update monitors through $\iota_r(\xi)$, leaves new monitors inactive and sets $P = \Sigma_{\text{upd}}$. The policy replaces C^{old} and discards its internal state and scheduling restrictions. Old monitors remain until their stops; undeclared objectives, including old liveness, are not preserved. The entry set is

$$Q_0 = \{\text{Init}(z) \mid z \in \text{Reach}(\text{CL}^{\text{old}})\}. \quad (2)$$

Requiring all reachable old states makes admission independent of waiting for a favorable old state. Selecting an admission predicate would be a different contract. In the cell, $Q_0 = \{q_A^0, q_B^0\}$ distinguishes the two holders.

Four kinds of induced transitions. For $\xi(i) = (v, s)$, component i uses e_i^v . The active-component product determines enabled ordinary uncontrollables $U(q)$, including those with unsafe outcomes, independently of updates and goals. For every $a \in \Sigma_{\text{upd}}$, impose

$$U(q) \neq \emptyset \implies \text{Post}(q, a) = \emptyset. \quad (3)$$

Here, *uncontrollable quiescence* means that no modeled uncontrollable event is enabled. It differs from transaction-based quiescence [16] and tranquility [31]; Section 5.2 states the additional execution assumptions. Global quiescence is an explicit modeling restriction, not a consequence of local state transfer. This precondition restricts update events only. Ordinary controllables may still coexist with uncontrollables, although suppressing them preserves winning decisions (Lemma 4.1). $\text{Post}(q, a)$ contains exactly the successors below; unmentioned coordinates remain unchanged. “Advance” applies $\mu'(t_r) = \eta_r(\mu(t_r), a)$ to the specified active monitors.

Ordinary event $a \in \Sigma_N$. If a belongs to the local event set of at least one active component and every active component having a can transition, take every synchronous-product outcome and advance every active monitor.

Component update ρ_i . If $U(q) = \emptyset$, order-enabled, and $\xi(i) = (\text{old}, s)$, set $\xi'(i) = (\text{new}, s')$ for each $s' \in g_i(s)$, delete ρ_i from P , and advance every active monitor.

Requirement deactivation $\text{stop}(r)$. If $U(q) = \emptyset$ and order-enabled, advance every active monitor except t_r , retain inactive monitors at $\perp$, set $\mu'(t_r) = \perp$, and delete $\text{stop}(r)$ from P ; t_r does not observe it.

Requirement activation $\text{start}(r)$. If $U(q) = \emptyset$, order-enabled, and $\xi \in \text{dom } t_r$, advance every active monitor, retain other inactive monitors at $\perp$, install $\mu'(t_r) = t_r(\xi)$, and delete $\text{start}(r)$ from P ; t_r begins online observation afterward.

On the upper cell path, start preserves old/empty A and new/holding B , installs inspection-pending in the new monitor and removes itself from P . The old and interval monitors stay active.

If a precondition is false, the successor set is empty; every nondeterministic outcome is included and environment-chosen. Let $\mathcal{Q}$ contain every state reachable from Q_0 through these transitions. *Safety and handover-ready goals*. An update state $q = (\xi, \mu, P)$ is safe, written $\text{Safe}(q)$, exactly when every active monitor is outside its error states. For $P = \emptyset$ and $z \in Z_{\text{load}}$, write $q \sim z$ when component states (without version tags) and all new-monitor states agree. The handover map κ selects such a full endpoint state z . Goals are

$$Q_{\text{goal}} = \{q = (\xi, \mu, \emptyset) \in \mathcal{Q} \mid \text{Safe}(q) \wedge U(q) = \emptyset \wedge \exists z \in Z_{\text{load}} : q \sim z\}. \quad (4)$$

At completion, all components and active endpoint monitors are new. Interval obligations end at the goal; new monitors continue after handover and must match. Compatible targets can differ in later behavior while satisfying the endpoint assumptions. At a quiescent goal, handover disables ordinary controllables and installs the controller state in $\kappa(q)$, outside Σ and the update rank; no uncontrollable can compete. A matching state with an enabled uncontrollable is not yet a goal.

State-dependent policies and finite completion. A memoryless update policy is a function $\sigma : \mathcal{Q} \setminus Q_{\text{goal}} \rightarrow 2^{\Sigma_c}$ selecting only events with nonempty successor sets.

The possible next states under this policy are

$$\text{Succ}_{\sigma}(q) = \bigcup_{u \in U(q)} \text{Post}(q, u) \cup \bigcup_{a \in \sigma(q)} \text{Post}(q, a). \quad (5)$$

The environment chooses any uncontrollable and any permitted-event outcome. An uncontrollable self-loop blocks updates by Equation (3) and permits a losing infinite run. The returned policy retains one controllable at a quiescent state.

Let $\text{Reach}_{\sigma}(Q_0)$ be the policy closure from Q_0 , stopping at goals.

Runs, winning, and witnesses. A run follows the policy successors and stops at a goal. It is *maximal* if infinite or ending at a goal or a state without successors. A single policy σ is winning for an FG-DUCS instance if, from every $q_0 \in Q_0$, (i) every reachable update state is safe, (ii) every reachable non-goal state has a successor, and (iii) every maximal update run reaches Q_{goal} after finitely many events. This *finite completion* counts observed transitions and assumes no fairness.

A *progress-ranking function* for a winning policy is $\text{rk} : \text{Reach}_\sigma(Q_0) \rightarrow \mathbb{N}$, with value 0 at goal states and satisfying $\text{rk}(q') < \text{rk}(q)$ for every non-goal state q and every $q' \in \text{Succ}_\sigma(q)$.

The FG-DUCS problem. The input is an eligible contract: fixed endpoints, paired components and local transfers, safety monitors and initializers, update precedence and handover targets. Decide whether one policy safely completes the induced game from every entry state under every permitted event and outcome. Return $(\sigma, \text{rk}, \kappa)$ covering all Q_0 , or a losing-region certificate.

3.3 Checking the Cell

S1 tabulates the complete component, controller and monitor LTSs and the policy: ordinary events m_A, m_B, j move to A , move to B and inspect at B , and all events, including the one-shot transfers ρ_A, ρ_B and requirement boundaries start $<$ stop, are controllable. With empty-only transfers $g_A = g_B = \{(e, e)\}$ and exactly one workpiece, the two paths $\rho_B; m_B; \text{start}; \text{stop}; j; \rho_A$ from A holding and $\rho_A; m_A; \text{start}; \text{stop}; \rho_B$ from B holding have safe singleton outcomes: start installs inspection-pending if B holds and inspection-clear otherwise, stop removes the inspection ban, and both paths finish with new versions and clear inspection at matching supplied targets. Their remaining lengths $6, \dots, 0$ and $5, \dots, 0$ strictly decrease, proving finite completion from both reachable old states (13 policy states and 11 edges). Replacing the two transfer commands by one with relation $g_A \times g_B$, with endpoints and requirements fixed, requires both stations empty, but every ordinary move, inspection and requirement boundary preserves exactly one workpiece; that command can never execute, stays pending and makes every goal unreachable.

Section 4 proves game guarantees; Section 5 states additional interpretation and execution assumptions.

4 Certifying On-the-Fly Synthesis

Lazy (FG-DUCS-OTF) avoids constructing the entire reachable game, while retaining every outcome of each queried event and every uncontrollable choice. It combines finite reachability reasoning with on-the-fly product exploration [7, 8, 30].

A *bucket* contains every outcome in $\text{Post}(q, a)$; emptiness means the event is disabled. With uncontrollables enabled, every UC outcome must win. Otherwise, one nonempty controllable bucket suffices if all its outcomes win. Lazy generation postpones other choices; Lemma 4.1 permits omitting controllable queries at UC-enabled states.

4.1 Lazy Successor Generation

Algorithm 1 builds a reachability attractor on the fly [5]. It tracks discovered states V , states with all UC candidates queried $X \subseteq V$, and proved winners W . States in X may retain unqueried controls; W need not contain all winners.

Discover. Queries add every outcome to V . Goals enter W at rank zero; safe non-goals enter Open, served before retries.

Expand or retry. Expansion queries all UC candidates first. If none is enabled, it queries controls through the next nonempty bucket or exhaustion, retaining an unfinished cursor in Retry. Candidates include local events and order-enabled pending commands, covering every enabled event; any fixed finite order is correct.

Promote. An expanded safe state's *proving set* is the complete UC-successor union if UC is enabled, or one nonempty control bucket at quiescence. When this nonempty set lies in W , the state wins with fixed rank one above its maximum successor rank. Reverse incidences notify predecessors of new winners and trigger reconsideration of recorded buckets. Failure requires exhausted candidates and a promotion fixed point.

On the cell's A -holding path, promotion works backward from the rank-zero goal. Inspection self-loops provide no lower-rank successor; nondeterministic buckets require every outcome to win.

Once all entries win, extraction retains one lower-rank control bucket at each reached quiescent non-goal and no controls elsewhere. Its choice can differ from promotion and need not yield shortest paths.

Algorithm 1: Lazy

Input: Typed update contract (Section 3)

Output: $(\sigma, \text{rk}, \kappa)$ or (no solution, L)

```

1  $V \leftarrow Q_0$ ;  $X, \text{Retry} \leftarrow \emptyset$ ;  $W \leftarrow V \cap Q_{\text{goal}}$ ;
2  $\text{Open} \leftarrow \{q \in Q_0 \mid \text{Safe}(q) \wedge q \notin Q_{\text{goal}}\}$ ; set  $\text{rk}(q) \leftarrow 0$  for  $q \in W$ ;
3 while true do
4   Insert queued goals into  $W$  with rank zero, once per goal;
5   while some safe  $q \in X \setminus W$  has a nonempty proving set  $S \subseteq W$  as defined under Promote do
6     Retain  $S$ ; set  $\text{rk}(q) \leftarrow 1 + \max_{s \in S} \text{rk}(s)$ ; add  $q$  to  $W$  and remove it from Retry;
7     Update reverse-incidence counters and reconsider affected predecessors;
8   if  $Q_0 \subseteq W$  then return  $(\sigma, \text{rk}|_{\text{Reach}_\sigma(Q_0)}, \kappa)$  constructed from the goal witnesses and retained lower-rank
    choices;
9   if  $\text{Open} \neq \emptyset$  then
10    Remove  $q$  from Open, add it to  $X$ , and query all uncontrollable candidates;
11    if a nonempty uncontrollable successor set exists then
12      Clear  $q$ 's controllable cursor;
13    else
14      Initialize  $q$ 's controllable cursor; query to the next nonempty bucket or exhaustion;
15    Insert  $q$  into Retry iff  $q \notin W$  and its cursor has remaining candidates;
16  else
17    if  $\text{Retry} = \emptyset$  then return (no solution,  $L \leftarrow V \setminus W$ );
18    Remove  $q$  from Retry; query to the next nonempty bucket or exhaustion; reinsert iff  $q \notin W$  and its
    cursor has remaining candidates;
19  Queue newly discovered goals for rank-zero insertion; put newly discovered safe non-goals in Open;
  
```

LEMMA 4.1 (UC PRIORITY). *Disabling controllable events at UC-enabled states preserves the existence of a policy for finite safe completion.*

PROOF. A winning policy must handle every UC outcome. Disabling optional controls removes runs and creates no deadlock while a UC event remains enabled. Every surviving maximal run was already a maximal run of that policy, so it still completes safely. Conversely, the reduced policy is admissible in the original game because controllers may disable those events. $\square$

LEMMA 4.2 (SEARCH COVERAGE). *At loop boundaries, every state in X has all UC candidates queried; every expanded unproved quiescent safe non-goal with candidates left is in Retry ; and every discovered safe non-goal outside X is in Open .*

PROOF. Initialization at entry states establishes coverage. Discovery queues each new safe non-goal; only expansion removes it, after querying all UC candidates. At quiescence, expansion and retries consume the cursor through a nonempty bucket or exhaustion, retaining every nonwinner with candidates left. Promotion removes only winners. These are the only changes to those sets; finite propagation drains before either return. $\square$

4.2 Correctness and Certificates

The state records versions, monitor memory and pending commands.

Lemma (memoryless policies suffice). If any history-dependent policy wins this finite game, a state-based policy wins. *Proof.* Starting with goals, attractor layers add safe states whose nonempty proving sets lie in previous layers. Choosing those sets gives a state-based policy with decreasing layer index. Outside the fixed point, the environment can keep an outcome outside after every history, or force an unsafe state or deadlock, defeating any history-dependent policy (S2).

Lemma (monitor intervals). Along every update prefix, each active monitor is its inherited or initialized state advanced over exactly the labels in its declared active interval. *Proof.* Entry inherits its old states and initializes interval monitors. Ordinary events and transfers advance all active monitors once. Stop removes its monitor before observation, while others advance. Start advances existing monitors and installs the new one after its boundary. Pending commands enforce one-shot execution and consistent versions/activity. Induction gives the stated interval; safety makes it non-error, and goal matching preserves new-monitor states into the endpoint.

THEOREM 4.3 (TERMINATION, SOUNDNESS, AND COMPLETENESS). *For every eligible finite FG-DUCS instance with Equation (3) and quiescent goals in Equation (4), Algorithm 1 terminates. A returned $(\sigma, \text{rk}, \kappa)$ keeps every active monitor non-error, preserves every uncontrollable event and every outcome of each policy event, reaches Q_{goal} finitely without deadlock, and gives each reached goal a compatible target in Z_{load} for the supplied game. It returns no solution iff no winning policy covers Q_0 in that supplied game.*

PROOF. *Termination.* The finite input induces finitely many states and candidates. Each state is discovered and expanded at most once; each retry consumes a new candidate, and each promotion adds a state to W once. Reverse incidences are processed on those promotions, so every worklist drain is finite. Eventually all entries win or both exploration queues empty.

Soundness. Induction on promotions gives complete successor sets with smaller retained ranks: expansion supplies all uncontrollables, or the selected nonempty controllable bucket supplies all outcomes. A queried bucket never changes, and ranks are fixed on first promotion, so this evidence persists. If all entries win, their policy closure is safe and every non-goal has a successor. Strict decrease of the natural-number rank excludes infinite runs; nonempty successors exclude non-goal sinks. Thus every maximal run reaches a goal within its initial rank many events. Goal recognition uses modeled uncontrollable quiescence and supplies each compatible $\kappa(q)$.

The monitor-interval lemma above supplies the interval invariant; goal matching preserves components and new-monitor states into the verified endpoint.

Completeness. At failure, promotion has reached its fixed point. An entry lies in $L = V \setminus W$, and L contains no goal. By Lemma 4.2, empty Open ensures every safe non-goal in L was expanded. With uncontrollables, complete expansion and exhausted propagation imply an uncontrollable outcome

in L ; otherwise promotion would apply. Without uncontrollables, empty Retry gives every enabled controllable bucket, each with an outcome in L for the same reason. Against any policy, the environment keeps choosing these outcomes; permitting no event produces a deadlock. The resulting run reaches an unsafe state, deadlocks outside a goal, or remains in L forever. Each contradicts winning. Hence a winning policy rules out failure; termination then forces success.

The losing-region strategy defeats history-dependent policies after every history; success supplies a ranked state-based policy. Lemma 4.1 preserves winning in the original supplied game. $\square$

Checking the returned evidence. For WIN, let $D_\sigma = \text{Reach}_\sigma(Q_0)$. The checker requires exactly D_σ as the rank domain, safety throughout it and, at every non-goal, enabled selected controls, nonempty successor sets inside it, and strict rank decrease for every outcome. At goals it checks rank zero, UC quiescence and compatible $\kappa(q) \in Z_{\text{load}}$, with $\text{dom } \kappa = D_\sigma \cap Q_{\text{goal}}$; omitted policy entries are empty.

For LOSS, $L \subseteq \mathcal{Q}$ must include an entry state and exclude all goals. Every safe $q \in L$ must have some UC outcome in L when UC is enabled; otherwise every enabled controllable must have an outcome in L . Unsafe states and non-goal deadlocks lose. These conditions certify search relative to shared Post and endpoint predicates [15]; Section 6 states the implementation trust boundary.

4.3 Exploration Cost and Independent Updates

With indexed predicates, output-sensitive successors and constant-time bookkeeping, work is linear in discovered states, queried candidates and returned outcomes. Each state is discovered, expanded and promoted at most once; each candidate is queried once and each outcome incidence is processed on promotion. Endpoint preprocessing is excluded. Java also sorts transitions and scans entries (cost model in S2). Direct-Full constructs the same terminalized game first. Worst-case exploration reaches the entire game, potentially exponential in component input.

Every discovered state and nonempty bucket belongs to the Direct-Full game. Both constructions share entries, candidates, Post and terminal predicates. Inductively, a lazy query starts at a shared nonterminal state and returns exactly outcomes that Direct-Full also generates. Neither expands unsafe states or goals; empty queries add no stored bucket. The next family gives a strict exploration gap, without implying a gap against symbolic representations.

An exponential separation family. For $n \geq 1$, take n components, each with one old and one new state and a singleton transfer. Every version and its endpoint controller admit a controllable tick self-loop, making both endpoints deadlock-free. There are no monitors, dependencies or uncontrollables; the unique new endpoint is available for handover. These inputs are eligible. Each update state is the subset of transferred components, and all 2^n subsets are reachable; Direct-Full discovers them all.

Under Lazy's default order, pending transfers precede ordinary events in this family, so expansion follows one n -step chain. Because Open precedes Retry, it reaches the sole goal after discovering exactly $n + 1$ states. Backward propagation then assigns ranks $0, \dots, n$ and proves the entry state before any alternative query. The ordinary self-loops are losing choices under finite completion; both methods use the same terminal goal. Thus the separation concerns exploration of one supplied game.

5 Interpreting and Executing a Checked Policy

5.1 Interpreting Requirement Initializers

A requirement initializer assigns monitor memory when that requirement starts. Residual soundness connects this memory to the intended history before activation. Let SafePref_r be the prefix-closed language of requirement r , and let $H_r(\xi)$ contain all admissible activation histories consistent with the observed component state ξ . A new requirement's history includes start once; its initialized monitor reads only later events. Residual-sound initialization means

$$\underbrace{\{\omega : \eta_r^*(l_r(\xi), \omega) \notin F_r\}}_{\text{continuations accepted by the initialized monitor}} \subseteq \underbrace{\bigcap_{h \in H_r(\xi)} \{\omega : hw \in \text{SafePref}_r\}}_{\text{continuations safe after every admitted history}} \quad (\text{RS})$$

for every $\xi \in \text{dom } l_r$. Equality is exact; strict inclusion is conservative and may exclude feasible updates. An empty history set makes the right side universal. Synthesis checks initializer states and domains, but neither infers histories nor establishes their completeness.

Deriving RS in the cell. The declared pre-activation observation retains only start, so $H_{r_{\text{inspect}}}(\xi) = \{\text{start}\}$ for every admissible cell tuple. The reference monitor starts at c , unchanged by start; the right-hand side of RS is therefore its non-error continuation language from c . If B is empty, initialization at c gives equality. If B holds, initialization at d is conservative: before the first j or m_B , d additionally forbids m_A ; either j or m_B brings the two initial states to the same state. Thus every word safe from d is safe from c , while the single event m_A witnesses strict inclusion. This is a proof for the declared observation scope, not recovery of pre-activation history.

A Boolean event-history variable, or *fluent*, records setting/resetting labels. For initially true p , a history making p false then true and the empty history end at the same value but differ under $\square p$. The evaluation distinguishes three history scopes:

A (activation-safe), for new requirements (NEW): restrict $H_r(\xi)$ to safe pre-activation histories.

E (entry-scoped), for interval requirements (UPD): start the reference at update entry.

H (history-inclusive, counting pre-activation violations): count pre-activation violations too; this scope is not claimed. Each chosen scope must match the intended requirement (S3); synthesis alone does not establish that interpretation.

5.2 Conditions for Execution

An execution adapter connects the policy to a running system through the following observations and operations.

Linking the phases. The linked model connects disjoint old/update/new phases by entry $z \mapsto \text{Init}(z)$, policy edges Succ_σ and handover $q \mapsto \kappa(q)$. Entry and handover add no game event.

Let α be a phase-disjoint observation map.

A1 (endpoints): actual endpoint observations map to same-labeled edges of $\text{CL}^\vee$ and preserve its component and monitor states.

A2 (update observations and progress): at a non-goal, filtering the complete environment event set disables every controllable event outside $\sigma(q)$ and disables no uncontrollable event; every remaining event/outcome maps into $\text{Post}(q, a)$ and is reported with its successor identity. The resulting set is nonempty and a next event is eventually observed. This progress premise assumes no fairness among event choices.

A3 (commands and observation): each command starts at uncontrollable quiescence, finishes atomically and failure-free in finite time, and reports one modeled transfer or monitor-lifecycle

transition with its successor identity. Internal τ steps, unobservable outcomes and divergence are outside the model.

A4 (boundaries): entry atomically observes the actual reachable old state z and switches to $\text{Init}(z)$; at a quiescent goal, ordinary controllables are disabled and handover installs its selected controller state and switches control. Each boundary is a finite, failure-free linearizable operation preserving the specified component and monitor states [13].

THEOREM 5.1 (CONDITIONAL TRACE LIFTING). *For a returned WIN certificate $(\sigma, \text{rk}, \kappa)$ of an eligible contract, every execution satisfying A1–A4 maps through α to the linked model. From an observed entry $\alpha(x_0) = \text{Init}(z) \in Q_0$, the update reaches quiescent handover within $\text{rk}(\text{Init}(z))$ observed game events and then hands control to the compatible endpoint. With complete activation histories as in Section 5.1 and RS for the supplied initializers, each declared active interval satisfies its requirement/residual interpretation. Agreement with application intent remains an input obligation.*

PROOF. A1 maps the old prefix into its monitored endpoint, and A4 maps entry into Q_0 . A2 preserves every UC move, disables controls outside the policy and maps each report to a complete Post bucket; A3 makes each command one finite observed step. Induction keeps observations in the safe certificate domain with strictly decreasing rank. A2 excludes stalling, so at most the initial rank many events reach a goal. A4 then hands control to its endpoint finitely; A1 maps the new continuation. Theorem 4.3 preserves each initialized monitor, and RS makes its continuations legal after every admitted activation history. $\square$

This execution guarantee requires an adapter satisfying A1–A4; we have not evaluated such an adapter.

6 Evaluation

Base imposes no interval requirement; R1 restricts actions between requirement boundaries; R2 relates those boundaries to component transfers. **RQ1 addresses L2:** do independent reconstruction and certificate/Link checks validate safe finite completion? **RQ2 addresses L1:** when do local transfer and requirement boundaries enable completion that a global step prevents? **RQ3 addresses L3:** how much exploration does Lazy save, and where do savings stop as the game grows?

Implementation. Our MTSA extension [9] checks certificates [20] relative to its parser, Post, monitors, transfers and endpoints. Link checks abstract old–update–new composition, entries and selected targets. The explicit checker builds successors separately but shares frontend and endpoint/goal code; the restricted raw-LTS oracle also parses independently. All implementations share authorship and AI assistance. None validates runtime conformance.

Constructing the benchmark inputs. Nine adapted inputs come from eight DUCS sources; two PC sizes share one. We declare paired components, transfers, endpoint formulas and Base/R1/R2 intervals. The frontend derives boundaries and initializers; each JVM verifies and fixes endpoints with MTSA’s GR backend. NEW uses fluent lookup, UPD post-entry constants; all reachable new closed-loop states are handover targets. Railcab retains both transfer outcomes; PC changes tool-result controllability; Surveillance changes battery transitions/safety; GSM/R2 constrains decode boundaries before ENV transfer. Table 1 and S4 retain counts and mappings. Source-problem equivalence is unestablished.

Fixed exploration variants. All variants share entries, Post/goal and checks; unsafe/goal states are terminal. At UC-enabled states OTF omits controllable buckets, while Direct-Full constructs them but applies the same UC-prioritized winning condition (Lemma 4.1). Otherwise Lazy queries one controllable bucket at a time and stops when all entries win; Update-first prioritizes all updates;

Table 1. The nine adapted inputs. Old/new requirements are shared across variants; interval counts are Base/R1/R2.

Input	Comp.	Entries	Old/New	Interval B/R1/R2	T	U
GSM	1	8	1/1	0/1/2	8	3
Industry	3	131	3/3	0/3/6	16	9
MetaSocket	2	4	1/1	0/1/2	3	4
PowerPlant	2	10	4/3	0/3/7	4	9
PC Arms=1	1	9	6/6	0/6/12	7	13
PC Arms=2	2	126	12/12	0/12/24	14	26
Railcab	4	22	5/7	0/7/11	11	16
Surveillance	2	112	7/7	0/7/14	16	16
Workflow	5	28	5/6	0/6/11	12	16

T counts declared transfer pairs before fluent observers; U counts transfer, stop and start commands. Entries agree across completed variants; PC Arms=2/R2 times out and its requirement counts come from its input log. Only Railcab has branching transfers. These nine inputs declare no explicit precedence; the separate cell witness requires start < stop; R2 uses interval monitors.

Eager queries all controllable buckets on expansion. Direct-Full builds the whole reachable game first.

Lazy’s default order is stop–transfer–ordinary while stops remain, then transfer–ordinary while transfers remain, with other updates last. Otherwise, start comes first only when a handover target matches the components and current or initialized new-monitor states. Neither order minimizes exploration in general.

State priority uses endpoint mismatch and pending-command counts; LIFO exploration precedes FIFO retries (tie-breaking in S2). Direct-Full is our same-game implementation, not an external solver. Order sensitivity appears in the saved trials: Update-first decides 24 inputs and is faster than Lazy on 8/24 completed pairs; Eager decides 17. The separation family in Section 4.3 is constructed for Lazy’s default order. Randomized orders and objective-preserving translations to external solvers have not been evaluated.

Design and measures. RQ3 retains all 27 instances (nine models, Base/R1/R2), comparing the four same-game variants above and the separate Legacy reference below.

Trials run sequentially on Windows 10, Xeon W-2265, 256 GB RAM, Temurin 17.0.20.1, with a 64 GiB heap and 1,200 s limit. Each of 135 instance–method cells runs once; valid decisions receive four more trials. Failures are not repeated. Medians and min–max require five consistent completions. Table 4 retains original interruptions and separate timeout follow-ups.

We measure states, queries, transitions, solver time, sampled peak process working set and policy size. Same-game solver time includes checking but excludes frontend, startup and endpoint preparation; Legacy uses its native timer. Total JVM time includes those costs; endpoint time is also recorded separately. Mac pilot times are excluded.

6.1 RQ1: Model-Level Validation

Answer. Lazy and Direct-Full agree with all independent raw-LTS expectations. Table 2 separates units and trust boundaries.

Six target/quiescence fixtures also match the raw-LTS reconstruction. All 138 frontend lookup exports match 1,587 reconstructed entries (900 non-error, 687 ERROR). These qualitative checks are outside performance statistics.

Table 2. Model-level checks and trust boundaries. The jobs share 46 fixtures; they are not independent applications.

Check	Result and unit	Separate implementation / remaining boundary
Raw-LTS fixtures	$46 \times 2 = 92$ jobs: 36 WIN, 48 LOSS, 8 invalid	Python separately parses and constructs graphs, ranks and losing closure. Invalid inputs omit events, deadlock or suppress UC.
Certificates and links	All 84 eligible jobs pass certificate and exhaustive checks; all 36 WIN pass Link	Separate explicit successors; shared front-end and endpoint/goal code.

Under Section 5.1’s scopes, independent exploration of monitors and observers with one-shot updates checks all 138 NEW occurrences. Equal fluent vectors yield equal continuation languages and agree with frontend lookup. Every admitted safe history reaches such a pair, proving RS inclusion. Exactness additionally needs nonempty actual safe histories, which remain unverified.

UPD starts a fresh reference with actual entry fluent values, consuming one entry marker (not a game event). The 89 R2 initializers reference only update-stage fluents whose commands cannot precede entry. The 46 R1 occurrences have $\Box(G \Rightarrow \neg A)$: G starts false and changes only on excluded pre-entry update commands, while ordinary-action predicates reset at entry. Thus entry valuations accept the same continuation language as the constant initializer. Static formula, label, reset and DFA checks establish exact entry-scoped RS for these 135 UPD occurrences; S3–S4 retain derivations. A1–A4 remain unvalidated.

6.2 RQ2: Can Local Choices Change Feasibility?

Answer. Eight files yield ten WIN/four LOSS, matching independent expectations: the cell separates local/joint transfer, an observed-branch witness requires both continuations, and a boundary witness needs alignment between stop/start. Their neutral controls win both ways (finite contracts in S1). These establish existence, not separation from arbitrary encodings.

Merging inherited contracts. Merging transfers, requirement boundaries, or both changes 0/54 Base and R1 decisions (E1); R2 requirements name individual events and are excluded. Inherited raw transfers are total and omit the constructed examples’ explicit startup/calibration phase; totality alone does not imply safe joint transfer. States decrease in 44 comparisons and stay equal in 10. The transformations reproduce the cell’s transfer LOSS, but boundary merging still permits the ordinary gap forbidden by the blocked fixture (E4-1). Corrected Cell_n , $n = 2, \dots, 10$, wins locally/with merged boundaries and loses with merged transfers/both (E4-2). Initial-state-only transfers yield 0 fine WIN / merged LOSS, 6 both-LOSS and 2 both-WIN pairs, with 10/18 incomplete (E5): enabled UC requests can prevent quiescent transfer. These are contract diagnostics, not DUCS encodings (S4).

Constructed update mechanisms. Across 55 main constructed comparisons, fine and merged contracts yield 33 WIN/LOSS, 11 LOSS / LOSS and 10 WIN / WIN pairs, with 1 unresolved PC2 comparison; these counts establish existence, not prevalence. All 70 Rolling cells satisfy WIN iff $k \leq n - m$, where k replicas restart together and at least m of n must remain ready; all 15 Canary pairs separate recoverable local transfers from a product transfer with adversarial outcomes. Policy separates requirement-specific boundaries and DB-Rolling separates secondary-first from joint replacement,

whereas singleton-boundary Rolling+Audit and backpressure Threads retain their null results (Table 3). These mechanisms supplement E1’s unchanged decisions; S4 keeps scale extensions, assumption controls and earlier failed inputs separate from the main comparison population.

Table 3. Constructed mechanisms. W/L means WIN/LOSS; comparisons hold each pair’s remaining contract fixed. Parameters and all controls are in S4.

Family	Changed operation	Fine	Merged
Cell _n	Joint transfer	W	L
Rolling	Restart all replicas	W	L
Canary	Joint transfer with all outcomes	W	L
Policy	Joint requirement boundaries	W	L
DB-Rolling	Joint transfer; no-slack control	W / L	L / L
Rolling+Audit	Transfer / boundary merging	W / W	L / W
Threads	Backpressure / saturation	W / L	W / L
PC2-Rolling	Joint transfer	W	TO*

Rolling compares single-replica with all-replica restart where the former can win; the threshold above also includes losing batch sizes. *A separate invariant establishes joint-transfer impossibility; the solver remains unresolved.

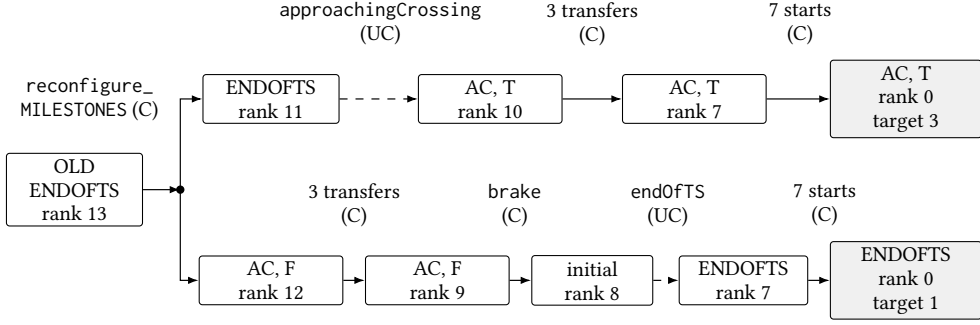
PC2-Rolling: distinct evidence. Fine-grained Lazy returns WIN for the supplied recalibrating two-arm plant. All checked reachable old states admit both raw transfers, but their joint successor calibrates both arms and immediately violates interval availability, proving joint-transfer impossibility. The merged solver and fine Direct-Full nevertheless time out at 7200 s/200 GiB; these are not losing decisions.

Following an application policy. Figure 2 retains both outcomes of a Railcab Base transfer with different continuations and targets. R1 bans braking after old stops; saved policies instead brake earlier or complete without braking. Both exports cover all entries and pass certificate/Link checks. The illustrated monitor permits crossing checks only after approach without a last emergency brake: the branches initialize different memories and match different endpoint states. S4 gives the full tuples; these qualitative exports add no timing or runtime validation.

What a losing contract means. Industry/R1 loses at 42 of 131 entries by independent exhaustive checking. Figure 3 shows a reachable safe quiescent state with no supplied target: GF1/TOR validation violates NEW; the safe DSD1 validation admits a rejection cycle, while disabling it deadlocks. The full losing fixed point, not this one path, excludes every update order. R1 forbids validation between old stop and new start. DUCS instead synthesizes continued new-specification control [24, Defs. 4.1–4.3]; its original build and aligned fork win on the source model. FG LOSS depends on supplied initializers, requirements and targets. Exempting GF1 from R1 makes all 131 entries WIN via earlier paths before DSD1/TOR activation, not escape from the displayed trap; this separate Mac contract variant supplies no timing data.

6.3 RQ3: Exploration Savings and Limits

Answer. Lazy avoids states and successor queries in all 14 completed Direct-Full pairs. Within the fixed implementation and budget, it decides 26 inputs, Eager 17 and Direct-Full 14 (Table 4). Priorities remain mixed: update-first has lower median solver time on 8/24 completed pairs but has two additional timeouts. All paired decisions agree. Lazy returns LOSS on Industry/R1 and times out on PC Arms=2/R2. The latter also times out at 3,600 s; its limiting resource remains unknown. Solver and total-JVM ratios measure different costs.



C: controllable; UC: uncontrollable (dashed). Shaded boxes: handover targets.

Fig. 2. Both outcomes of one Railcab/Base transfer after five old stops and before any new start. AC is approaching-crossing; T/F records whether that event occurred since endOfTS. Boxes show milestone state and rank; targets identify supplied new-controller states. Only single-path sequences of three transfers or seven starts are contracted. Every branch of this 26-state, 25-edge fragment is retained.

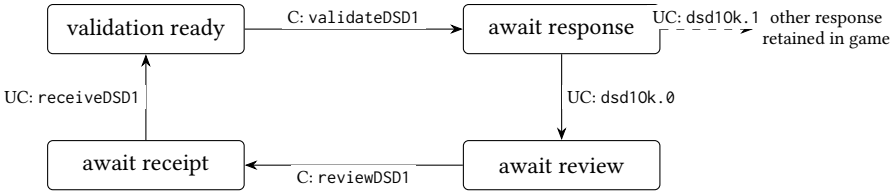


Fig. 3. The safe core of a losing region in Industry/R1 after all updates. The solid cycle combines the only safe control choices with an environment choice that can repeat rejection forever. At each of the four safe states, the other controllable events, validateGF1 and validateTOR, lead to monitor error. C/UC denote controllable/uncontrollable events. The dashed UC outcome is retained; the environment need not choose it. The changed contract enables an earlier update path, not an escape from this region.

Table 4. All 27 instances per method. TO denotes one capped trial, not a timing median. Ratios summarize heterogeneous completed pairs; values above one favor Lazy.

Method	WIN/LOSS/TO	Pairs	Faster than Lazy	Solver ratio	Total JVM ratio
Lazy	25/1/1	—	—	—	—
Eager	17/0/10	17	0 2.04 [1.18–4,575.05]	1.60 [1.00–144.85]	
Update-first	23/1/3	24	8 1.07 [0.19–39.76]	1.00 [0.32–6.10]	
Direct-Full	14/0/13 [†]	14	0 9.14 [1.58–2,123.81]	2.27 [1.00–130.66]	

TO: timeout; no OOM or unmeasured case for these four methods. [†] Workflow/R2 includes a separate same-setting timeout after its original interruption; both remain in S4. Ratios are comparator/Lazy: median [min–max] across completed pairs using five-run medians. Pairs counts completed pairs; Faster counts those with lower comparator solver time. JVM includes frontend, startup, endpoints and output; solver includes checking. Unresolved costs receive no ratio.

Full per-input solver medians remain in S4; the ranges below retain absolute cost scale.

For 12 capped Direct-Full attempts, timeout/Lazy-median total-JVM ratios are 2.12–910.06 (median 114.72), including Workflow/R2’s separate follow-up; the interruption and both-timeout PC

case are excluded. These single-attempt lower bounds are not solver ratios or Direct-Full medians. Across 14 completed pairs, first-trial state/query ratios have medians 5.00/18.73. Lazy solver medians span 0.0136–562 s (full absolute counts/times in S4).

Separating construction choices. E2 uses a separate build to add full construction with UC pruning, preserving Post/goal. The two factors are construction (full/OTF) and pruning (off/on): Direct-Full is full/off, E2 full/on, Eager/Lazy OTF/on; OTF/off is unimplemented. Mac E2 yields 16 WIN, 1 LOSS and 10 TO; states decrease against Direct-Full in 10/16 comparable cells. Eager counts agree in 14/16; GSM/R1 and PowerPlant/R1 differ. On PC1/Base, Eager and E2 both discover 407019 states, versus Lazy’s 83; separate Xeon solve-plus-check times are 26.016 s (Eager, five-run median) and 158.667 s (E2, one trial). Xeon E2 yields 15 WIN/12 TO, including Industry/R1 despite Mac’s LOSS. S4 keeps the state/time tables and budgets separate; this incomplete comparison establishes neither equal state sets nor a factorial causal decomposition.

Memory and returned policies. Lazy’s completed five-run median peak working sets span 0.102–7.36 GiB (median 0.837). Sampled process memory includes JVM/native overhead; it establishes no heap or general memory dominance, and Eager/Update-first sometimes use less (S4). The 25 WIN policies have 8–1,071 states (median 55) and maximum ranks 4–34 (median 19), bounding events rather than downtime or optimal length. Counts agree over five repeats; LOSS/TO returns no winning policy.

Separate Legacy reference. This fork’s DUCS path uses a GR objective without fixed-endpoint handover: 9 matching WIN, 13 OOM, 5 snapshot-cap cases (N/M). On original-source inputs, the lab build and fork return 21 and 16 controllers: 15 first-trial sizes match, one differs; 5 inputs fail in parsing or instrumentation. S4 retains repetition variation. At 200 GiB and 3600 s, all 18 separate single trials remain undecided (11 OOM, 7 author-instrumentation resource-profile stops), so the original heap cap alone does not explain the absence of decisions. Logged intermediate products before the profile stops reach about 0.7–5.3 million states; these are neither returned controllers nor evidence of Legacy’s inability to solve its different objective (S4).

Scaling limits. All four methods finish the independent-update family for $n = 2, \dots, 20$. At $n = 20$, lazy and update-first discover 21 states, eager 211, and Direct-Full 1,048,576. Their five-run median solver times are respectively 0.0354, 0.0318, 0.0625 and 853 s. This constructed family exhibits the predicted gap; the Travel family below exposes limits.

Controlled families vary fanout, transfer outcomes, UC events/outcomes, requirement count or precedence. All four methods agree on 31 WIN and two LOSS across 33 inputs (132 single-run jobs). They cover broader features than the applications, but each family may change multiple graph measures; S4 gives their definitions and counts.

Travel has an agency and N amenities. Each amenity’s K UC choices lead to $0-K-1$ selection steps, coupling branching, length and size. Adaptations use direct/reserved purchasing, controllable admission, idle-only transfers and omitted source monitors; no trace equivalence is claimed.

Figure 4 retains all outcomes. Lazy, Eager and Update-first each decide 20 of 48 settings; Direct-Full decides 18. The OTF variants share all resource outcomes. Only $(N, K) = (4, 4), (4, 6)$ extend their budget-decided range over Direct-Full, which times out on both. On 18 completed pairs, Lazy/Direct-Full state ratios are 3.4–46.2%; Direct-Full/Lazy ratios of five-run median solver times are 1.49–34.21 (median 3.93). Travel (6, 1) still times out for Lazy and Direct-Full at 200 GiB and 7,200 s (about 79 GiB peak RSS), so its limit is not solely the original heap cap.

Lazy / Eager / Update-first									Direct-Full								
$N \backslash K$	1	2	4	6	8	12	16	20	$N \backslash K$	1	2	4	6	8	12	16	20
1	W	W	W	W	W	W	W	W	1	W	W	W	W	W	W	W	W
2	W	W	W	W	W	W	W	W	2	W	W	W	W	W	W	W	W
4	W	W	W	W	T	T	T	T	4	W	W	T	T	T	T	T	T
6	T	T	O	O	T	T	T	T	6	T	T	O	O	T	T	T	T
8	T	T	T	T	T	T	T	T	8	T	T	T	T	T	T	T	T
12	T	T	T	T	T	T	T	T	12	T	T	T	T	T	T	T	T

Fig. 4. All 192 first-trial Travel outcomes, indexed by amenities N and choices K . The three left-panel methods coincide at every setting, each with 20 WIN (W), 26 timeouts (T) and 2 OOM (O); Direct-Full has 18W/28T/2O. Every trial uses a 64 GiB heap and a 1,200 s whole-JVM budget. No unresolved time is imputed.

Extended-budget trials. At 7,200 s on the same Xeon/JAR, 27 planned single trials (64/200 GiB) yield 7 WIN, 1 LOSS, 16 TO, 1 OOM and 2 skips. All 5 comparable Direct-Full decisions match fixed-budget Lazy (Industry/R1 LOSS; PC1/R1 and Workflow/R2 WIN included); 3 further WIN concern Travel (4, 8), unresolved under the fixed budget. There, JVM/solver seconds are 1,636/756 for Lazy, 1,655/718 for Update-first and 2,036/961 for Eager. Workflow/R2 Direct-Full finishes in 1,387 s with 1,774,711 states. At 200 GiB, 9 Direct-Full application cells time out (peak RSS 159–213 GiB) and Surveillance/R1 exhausts its Java heap after 5,271 s; all 10 were unresolved at 64 GiB too. Fixed-budget Lazy decides 9 of these, with five-run medians ≤ 37.1 solver seconds and ≤ 7.4 GiB peak RSS. Lazy’s PC2/R2 remains TO (187 GiB RSS). Both methods time out at Travel (6, 1); the preregistered monotone rule skips (6, 4). RSS measures process memory, not reachable-state cardinality or heap occupancy. S4 reports all outcomes and timing definitions.

On the same Xeon/JAR, Eager’s 200 GiB/7,200 s single trials return 3 WIN where Direct-Full remained unresolved at that heap, discovering about $(1.08\text{--}2.66) \times 10^7$ states with UC pruning (60–68,000 times Lazy); Industry/R1 returns LOSS with 94,046 states (2.4 times). All 4 decisions match fixed-budget Lazy; ratios concern decided pairs only. The other 6 trials yield 5 TO/1 OOM at about 209–212 GiB peak RSS, a process-memory measure (S4).

6.4 Threats to Validity

Shared authorship, AI assistance and semantics permit correlated faults; the independent parser handles restricted FSP. Application decoding/history completeness and A1–A4 remain unvalidated. A permanent UC loop can block update/handover. Completeness is relative to the supplied game; new-controller resynthesis and contract repair are outside scope.

Inherited inputs show no E1 separation; E5/Threads retain null and both-LOSS cases. Operationally motivated witnesses assume finite startup, reporting, recovery and election and establish existence, not prevalence. PC2’s joint-transfer invariant and solver TO are distinct evidence. E2 lacks OTF without UC pruning and does not isolate every runtime cost. Legacy has a different objective; original-source comparison retains 5 unprocessed conditions, Railcab size disagreement and repetition variation, while capture-profile stops are author instrumentation.

Completed-pair ratios exclude unresolved costs; S4 preserves every cell, interruption and follow-up. Faster Update-first cases, Industry’s contract sensitivity and unresolved Travel/PC settings limit the advantage. Sampling may miss memory peaks. Related fixtures/models/generators and repetitions are not independent applications; one machine limits generalization. Applications have

no explicit precedence and only Railcab has branching transfers. Direct-Full is an in-house same-game comparator. External objective-preserving, compositional/symbolic and randomized-order comparisons remain unevaluated; the separation applies to Lazy’s default order.

Use of generative AI. Generative AI use on this project began on June 26, 2026, with the OpenAI Codex desktop app; no earlier use occurred. Codex assisted result aggregation and Python/LaTeX tooling (June 26–29), Maven build repair (June 29), translation and terminology unification of an earlier draft (July 1), drafting of this manuscript (from July 15), OTF implementation fixes, tests, experiments and analysis (July 22–August 15) and scaling-experiment scripts (August 28); logged model identifiers are gpt-5.5 (June 26–July 1) and gpt-5.6-sol (July 15–August 28). From September 14, Codex with GPT-6 Astra (gpt-6-astra) assisted contract design, implementation, experiments, analysis, figures and prose, and desktop Claude Fable 5.1 assisted planning, the Windows driver/runbook, initial raw/source inspection and, later, the introduction/example draft; Claude wrote neither Java nor the renderer. Solely as writing and editing aids, Codex produced one mock review of an earlier draft (August 7, gpt-5.6-sol) and structure candidates with reviews of them (September 20), while ChatGPT (6 Pro, as shown in its interface; September 18–28) and Claude produced mock reviews, reading-comprehension checks and side-by-side comparisons of anonymous drafts; these are neither human-subject studies nor evidence of validity. Tests, certificates, raw-LTS oracles, primary sources and author review provide validation, with correlated faults possible. Experiment data were processed locally; anonymous manuscripts went to external services. Numbers use saved outputs, not initial AI readings. No generative AI was used beyond these systems and purposes; individual sessions are not itemized. Authors retain responsibility.

7 Related Work

DUCS combines global transfers and requirement-set boundaries to synthesize update and subsequent control [24, Defs. 4.1–4.3]; its new environment can encode intermediate stages [24, Sec. 4.2]. FG-DUCS generates a fixed-endpoint game from local transfers and individual boundaries. DUCS re-encoding effort and equivalence remain unevaluated. O-DUCS adds update priority and phase-separated analysis; evaluation remains future work [14].

The GR(1) bridge preserves old then new safety under new-environment safety assumptions; new justice guarantees depend on new justice assumptions [2]. Live Synthesis carries unfinished obligations forward [11]. Before/after/between scopes [10] and event-wise residual rewriting [26] precede RS, which quantifies over compatible histories.

State attraction completes a task and reaches configuration startup states within bounded transitions under weak invariance [25]. Dynamic-feature [29] and compositional synthesis [21] produce maximally permissive, controllable, nonblocking supervisors for one specification; modular control decomposes its restrictions [32]. Nonblockingness requires a completing continuation from every reachable state. FG-DUCS connects fixed endpoints and requires every maximal update run to finish. OTF-DCS requires a marked extension of every controlled finite trace [8, Sec. 3]; FG-DUCS requires finite quiescent handover without fairness. Comparing solvers requires an objective-preserving translation.

8 Conclusion

FG-DUCS synthesizes model-level policies between fixed controllers from local transfers, requirement lifetimes and handover targets. Lazy exploration saves construction subject to order, contract and resource limits. Execution adapters, relaxed quiescence and same-contract symbolic comparisons remain future work.

Data Availability. The anonymous repository [4] provides the complete MTSA-derived source tree with FG-DUCS extensions, models, experiment and analysis scripts, generated results, the split raw-output archive and supplementary material. Third-party dependencies are retrieved from official upstream sources by the supplied scripts. The README links each claim to its evidence and explains unresolved outcomes, raw-output packages, source builds and reproduction tiers. The tag fse27-submission identifies the submitted version.

References

- [1] Bowen Alpern and Fred B. Schneider. 1985. Defining Liveness. *Inform. Process. Lett.* 21, 4 (1985), 181–185. doi:10.1016/0020-0190(85)90056-0
- [2] Gal Amram, Shahar Maoz, Itai Segall, and Matan Yossef. 2022. Dynamic Update for Synthesized GR(1) Controllers. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, USA, 786–797. doi:10.1145/3510003.3510054
- [3] Jesper Andersson, Luciano Baresi, Nelly Bencomo, Rogério de Lemos, Alessandra Gorla, Paola Inverardi, and Thomas Vogel. 2013. Software Engineering Processes for Self-Adaptive Systems. In *Software Engineering for Self-Adaptive Systems II*, Rogério de Lemos, Holger Giese, Hausi A. Müller, and Mary Shaw (Eds.). Lecture Notes in Computer Science, Vol. 7475. Springer, Berlin, Germany, 51–75. doi:10.1007/978-3-642-35813-5_3
- [4] Anonymous FG-DUCS Authors. 2026. FG-DUCS Evaluation Artifact. Anonymous replication package. Submission identifier: tag fse27-submission.. <https://github.com/research-artifact-archive/research-artifact>
- [5] Franck Cassez, Alexandre David, Emmanuel Fleury, Kim G. Larsen, and Didier Lime. 2005. Efficient On-the-Fly Algorithms for the Analysis of Timed Games. In *CONCUR 2005 – Concurrency Theory (Lecture Notes in Computer Science, Vol. 3653)*, 66–80. doi:10.1007/11539452_9
- [6] Betty H. C. Cheng, Rogério de Lemos, Holger Giese, Paola Inverardi, Jeff Magee, Jesper Andersson, Basil Becker, Nelly Bencomo, Yuriy Brun, Bojan Cukic, Giovanna Di Marzo Serugendo, Schahram Dustdar, Anthony Finkelstein, Cristina Gacek, Kurt Geihs, Vincenzo Grassi, Gabor Karsai, Holger M. Kienle, Jeff Kramer, Marin Litoiu, Sam Malek, Raffaella Mirandola, Hausi A. Müller, Sooyong Park, Mary Shaw, Matthias Tichy, Massimo Tivoli, Danny Weyns, and Jon Whittle. 2009. Software Engineering for Self-Adaptive Systems: A Research Roadmap. In *Software Engineering for Self-Adaptive Systems*. Lecture Notes in Computer Science, Vol. 5525. Springer, Berlin, Germany, 1–26. doi:10.1007/978-3-642-02161-9_1
- [7] Alessandro Cimatti, Marco Roveri, and Paolo Traverso. 1998. Strong Planning in Non-Deterministic Domains via Model Checking. In *Proceedings of the Fourth International Conference on Artificial Intelligence Planning Systems (AIPS '98)*. AAAI Press, Menlo Park, CA, USA, 36–43. <https://faaai.org/Library/AIPS/1998/aips98-005.php>
- [8] Daniel Ciolek, Matias Duran, Florencia Zanollo, Nicolás Pazos, Julián Braier, Víctor Braberman, Nicolás D'Ippolito, and Sebastián Uchitel. 2023. On-the-Fly Informed Search of Non-Blocking Directed Controllers. *Automatica* 147 (2023), 110731. doi:10.1016/j.automatica.2022.110731
- [9] Nicolas D'Ippolito, Dario Fischbein, Marsha Chechik, and Sebastian Uchitel. 2008. MTSA: The Modal Transition System Analyser. In *2008 23rd IEEE/ACM International Conference on Automated Software Engineering*. IEEE, Piscataway, NJ, USA, 475–476. doi:10.1109/ASE.2008.78
- [10] Matthew B. Dwyer, George S. Avrunin, and James C. Corbett. 1999. Patterns in Property Specifications for Finite-State Verification. In *Proceedings of the 21st International Conference on Software Engineering (ICSE '99)*. ACM, New York, NY, USA, 411–420. doi:10.1145/302405.302672
- [11] Bernd Finkbeiner, Felix Klein, and Niklas Metzger. 2022. Live Synthesis. *Innovations in Systems and Software Engineering* 18, 3 (2022), 443–454. doi:10.1007/s11334-022-00447-5
- [12] Cristiano Giuffrida, Anton Kuijsten, and Andrew S. Tanenbaum. 2013. Safe and Automatic Live Update for Operating Systems. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '13)*. ACM, New York, NY, USA, 279–292. doi:10.1145/2451116.2451147
- [13] Maurice P. Herlihy and Jeannette M. Wing. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems* 12, 3 (1990), 463–492. doi:10.1145/78969.78972
- [14] Takanori Hirano, Takuto Yamauchi, Kenji Tei, and Shinichi Honiden. 2024. Dynamic Update Controller Synthesis using on-the-fly Exploration. *IEICE Technical Report* 123, 443 (March 2024), 76–78. In Japanese; non-peer-reviewed technical report. <https://ken.ieice.org/ken/paper/20240314Vcccd/>
- [15] Martin Hofmann, Christian Neukirchen, and Harald Rueß. 2016. Certification for μ -Calculus with Winning Strategies. In *Model Checking Software (Lecture Notes in Computer Science, Vol. 9641)*. Springer, Cham, Switzerland, 111–128. doi:10.1007/978-3-319-32582-8_8
- [16] Jeff Kramer and Jeff Magee. 1990. The Evolving Philosophers Problem: Dynamic Change Management. *IEEE Transactions on Software Engineering* 16, 11 (1990), 1293–1306. doi:10.1109/32.60317

- [17] Orna Kupferman and Moshe Y. Vardi. 2001. Model Checking of Safety Properties. *Formal Methods in System Design* 19, 3 (2001), 291–314. doi:10.1023/A:1011254632723
- [18] Claus Lewerentz and Thomas Lindner. 1995. Case Study “Production Cell”: A Comparative Study in Formal Specification and Verification. In *KORSO: Methods, Languages, and Tools for the Construction of Correct Software*, Manfred Broy and Stefan Jähnichen (Eds.). Lecture Notes in Computer Science, Vol. 1009. Springer, Berlin, Germany, 388–416. doi:10.1007/BFb0015473
- [19] Jeff Magee and Jeff Kramer. 2006. *Concurrency: State Models and Java Programs* (2 ed.). John Wiley & Sons, Chichester, UK. <https://uat.store.wiley.com/en-us/concurrency-state-models-and-java-programs-2nd-edition-p-9780470093559>
- [20] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. 2011. Certifying Algorithms. *Computer Science Review* 5, 2 (2011), 119–161. doi:10.1016/j.cosrev.2010.09.009
- [21] Sahar Mohajerani, Robi Malik, and Martin Fabian. 2014. A Framework for Compositional Synthesis of Modular Nonblocking Supervisors. *IEEE Trans. Automat. Control* 59, 1 (2014), 150–162. doi:10.1109/TAC.2013.2283109
- [22] MongoDB. [n. d.]. Upgrade to MongoDB Enterprise (Replica Set). MongoDB Database Manual, version 7.0. Accessed 2026-09-29. <https://www.mongodb.com/docs/v7.0/tutorial/upgrade-to-enterprise-replica-set/>
- [23] Leandro Nahabedian, Víctor Braberman, Nicolás D’Ippolito, Shinichi Honiden, Jeff Kramer, Kenji Tei, and Sebastián Uchitel. 2016. Assured and Correct Dynamic Update of Controllers. In *Proceedings of the 11th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS ’16)*. ACM, New York, NY, USA, 96–107. doi:10.1145/2897053.2897056
- [24] Leandro Nahabedian, Víctor Braberman, Nicolás D’Ippolito, Shinichi Honiden, Jeff Kramer, Kenji Tei, and Sebastián Uchitel. 2020. Dynamic update of discrete event controllers. *IEEE Transactions on Software Engineering* 46, 11 (2020), 1220–1240. doi:10.1109/TSE.2018.2876843
- [25] Anas Nooruldeen and Klaus Werner Schmidt. 2015. State Attraction Under Language Specification for the Reconfiguration of Discrete Event Systems. *IEEE Trans. Automat. Control* 60, 6 (2015), 1630–1634. doi:10.1109/TAC.2014.2358811
- [26] Grigore Roşu and Klaus Havelund. 2005. Rewriting-Based Techniques for Runtime Verification. *Automated Software Engineering* 12, 2 (2005), 151–197. doi:10.1007/s10515-005-6205-y
- [27] Daniel Sokolowski, Pascal Weisenburger, and Guido Salvaneschi. 2022. Change Is the Only Constant: Dynamic Updates for Workflows. In *Proceedings of the 44th International Conference on Software Engineering (ICSE ’22)*. ACM, New York, NY, USA, 350–362. doi:10.1145/3510003.3510065
- [28] The Kubernetes Authors. [n. d.]. Deployments. Kubernetes Documentation. Accessed 2026-09-29. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- [29] Sander B. Thuijsman and Michel A. Reniers. 2024. Supervisory Control for Dynamic Feature Configuration in Product Lines. *ACM Transactions on Embedded Computing Systems* 23, 5, Article 71 (2024), 25 pages. doi:10.1145/3579644
- [30] Stavros Tripakis and Karine Altisen. 1999. On-the-Fly Controller Synthesis for Discrete and Dense-Time Systems. In *Formal Methods (Lecture Notes in Computer Science, Vol. 1708)*. Springer, Berlin, Germany, 233–252. doi:10.1007/3-540-48119-2_15
- [31] Yves Vandewoude, Peter Ebraert, Yolande Berbers, and Theo D’Hondt. 2007. Tranquility: A Low Disruptive Alternative to Quiescence for Ensuring Safe Dynamic Updates. *IEEE Transactions on Software Engineering* 33, 12 (2007), 856–868. doi:10.1109/TSE.2007.70733
- [32] W. Murray Wonham and Peter J. Ramadge. 1988. Modular Supervisory Control of Discrete-Event Systems. *Mathematics of Control, Signals, and Systems* 1, 1 (1988), 13–30. doi:10.1007/BF02551233